



Software Project Report 2024

Sweet Management System

Done By:

Shams Imad Abd Al-Aziz (Sec. 1)

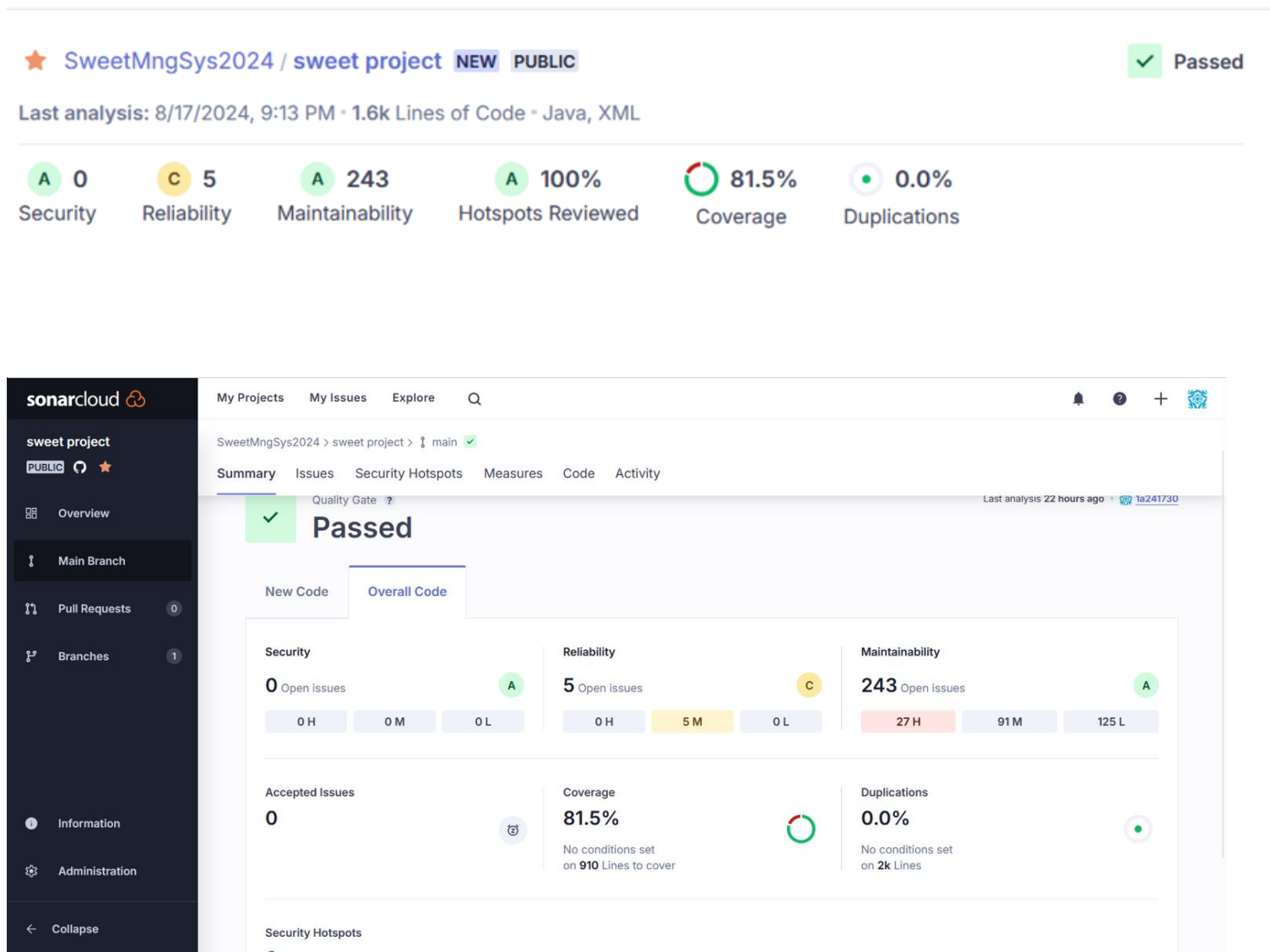
Bayan Futyan (Sec. 2)



Refactoring Phase

The first step we did is to connect our project in git hub with any static analyzer tool, and we use Sonar Cloud, to see the issues in our code and measure the coverage percentage, then refactor it to enhance it, and we use git hub action as CI/CD server.

And this is our code before any changes:





and this is how the git hub action start, by creating yml file to connect it with the Sonar, then show the status of the code, then I start refactoring the code.

Refactoring:

----- This bug appeared under reliability because we compared using `==` not using `"equals ()"`.



How we fix it:

```
774  
775 ✓ public boolean handleOrderStatus(String status2, String branch) {  
776     String status=status2.toLowerCase();  
777  
778     if(! (status.equals("on pending") || status.equals("on delivered") || status.equals("on processing") || status.equals("cancelled"))) return false;  
779  
780
```

This is the rate after resolving the bug. We solved 5 similar bugs.



----- for this bug indicating that should be rename method “applayDiscount”.

Consistency | Not identifiable

Rename method "applayDiscount" to prevent any misunderstanding/clash with field "applayDiscount".

Methods and field names should not be the same or differ only by capitalization
[java:S1845](#)

Software qualities impacted: Maintainability

☐ Open ☒ Shams Abd Al-Aziz ☒ Code Smell ☒ Blocker

Tags

confusing +

Line affected
L767

Effort
10 min

Introduced
4 days ago

```
public void applayDiscount(double minPrice, double discount,String branch) {  
  
    this.applayDiscount=true;  
    for(product prod: products)  
    {
```

Rename method "applayDiscount" to prevent any misunderstanding/clash with field "applayDiscount".

```
    }  
}
```



So, we rename it to:

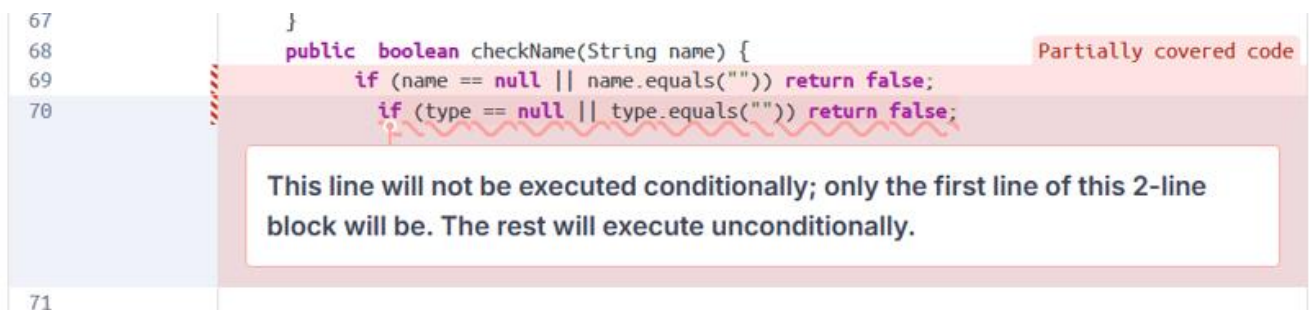
```
public void applyDiscountToProduct(double minPrice, double discount, String branch)
```

Under the same type of bugs, we rename the name of package:



```
1 package sweet.app;
2
3 import java.util.ArrayList;
4
5 public class Feedback {
6
7 }
```

----- for this bug we want to resolve, indicating that only the first condition in the block was being executed, while the subsequent lines were executed unconditionally, leading to unexpected behavior.





As we can see, the issue was resolved by combining the conditions into a single if statement, ensuring that all lines in the block are executed conditionally.

```
69
70  ✓      public boolean checkName(String name) {
71          if (isEmpty(name) || isEmpty(type)) {
72              return false;
73          }
74      }
```

----- for this bug, the word 'Jenin' has been duplicated, and this happened for another words.



So, to fix it:

```
private String jeninCity;
```

In the class constructor:

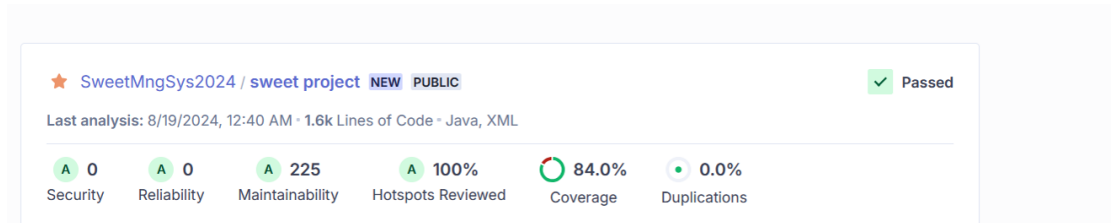
```
jeninCity="Jenin";
```

In the code:

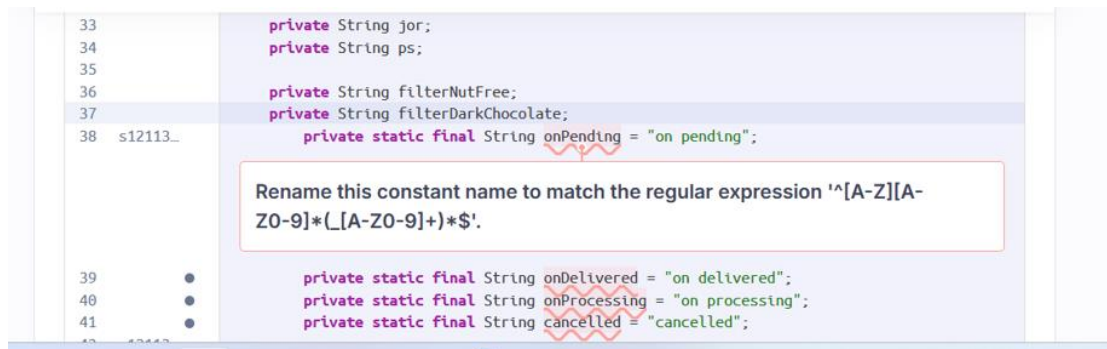
```
users.add(new User("bayan", "password123", "User", jeninCity));
```




And this is after the refactoring these bugs:



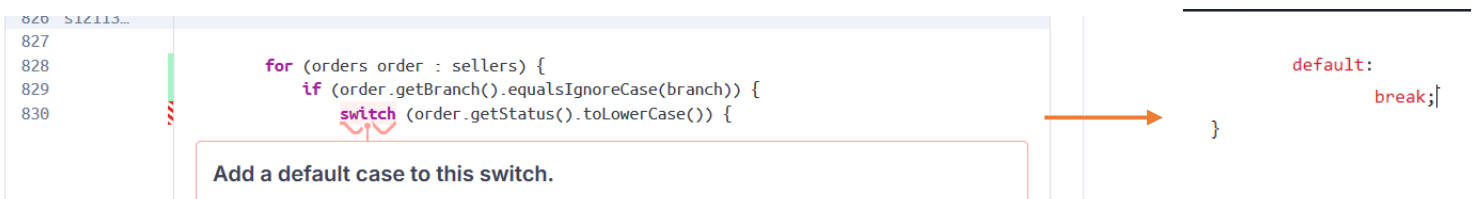
Also, we had this:



And we did this:

```
private String filterNutFree;
private String filterDarkChocolate;
private static final String ON_PENDING_LABEL= "on pending";
private static final String ON_DELIVERED = "on delivered";
private static final String ON_PROCESSING = "on processing";
private static final String CANCELLED = "cancelled";
```

----- for this bug, we want to resolve by add a default case to this switch.





And this is the status of
our project:

▼ Type

🐛 Bug	0
🔒 Vulnerability	0
💩 Code Smell	220

---- for this bug for this method inside the file MyApp.java has exceeded the allowed complexity limit, reaching 24 points while the permitted limit is 15 points.

Refactor this method to reduce its Cognitive Complexity from 24 to the 15 allowed. [🔗](#)

Cognitive Complexity of methods should not be too high [java:S3776](#)

Software qualities impacted: Maintainability

☐ Open ☒ Shams Abd Al-Aziz ☐ Code Smell ☒ Critical

Where	Why	How	Activity	More info
-------	-----	-----	----------	-----------

```
824      String status=status2.toLowerCase();
825
826 s12113_  if(!(status.equals(STATUS_ON_PENDING) || status.equals(STATUS_ON_DELIVERED) ||
827 s12113_  status.equals(STATUS_ON_PROCESSING) || status.equals(STATUS_CANCELLED))) return false;
828
829      for (orders order : sellers) {
830          if (order.getBranch().equalsIgnoreCase(branch)) {
831              switch (order.getStatus().toLowerCase()) {
832 s12113_  case STATUS_ON_PENDING:
833          if (status.equals(STATUS_ON_PENDING)) {
834 s12113_  handlePendingOrder(order);
835 s12113_  order.setStatus(STATUS_ON_PROCESSING);
836 s12113_  }
837          break;
```

brain-overload

Line affected
L823

Effort
14 min

Introduced
1 day ago

Open in



```
public boolean handleOrderStatus(String status2, String branch) {  
    String status=status2.toLowerCase();  
  
    if(!checkStatus(status)) return false;  
  
    for (orders order : sellers) {  
        if (order.getBranch().equalsIgnoreCase(branch)) {  
            processOrderByStatus(order, status);  
        }  
    }  
    return true;  
}
```

```
public boolean checkStatus(String status){  
  
    return status.equals(STATUS_ON_PENDING)||status.equals(STATUS_ON_DELIVERED)||status.equals(STATUS_ON_PROCESSING)||status.equals(STATUS_CANCELLED);  
}  
  
public void processOrderByStatus(orders order, String status)  
{  
    String orderStatus=order.getStatus().toLowerCase();  
    if(orderStatus.equals(STATUS_ON_PENDING)&&status.equals(STATUS_ON_PENDING)){  
        handlePendingOrder(order);  
        order.setStatus(STATUS_ON_PROCESSING);  
    }  
    else if(orderStatus.equals(STATUS_ON_PROCESSING)&&status.equals(STATUS_ON_PROCESSING)){  
        handleProcessingOrder(order);  
        order.setStatus(STATUS_ON_DELIVERED);  
    }  
  
    else if(orderStatus.equals(STATUS_ON_DELIVERED)&&status.equals(STATUS_ON_DELIVERED)){  
        handleDeliveredOrder(order);  
        order.setStatus(STATUS_CANCELLED);  
    }  
    else if(orderStatus.equals(STATUS_CANCELLED)&&status.equals(STATUS_CANCELLED)){  
        handleCancelledOrder(order);  
    }  
  
    return;  
}
```

So, here is how we fix this bug .by refactoring the handleOrderStatus method. We have separated the complex logic into smaller, more manageable methods like checkStatus and processOrderByStatus, which handle specific tasks, thereby reducing the cognitive complexity of the original method.



----- for this bug:

```
public boolean checkType(String type) {  
    if(type.equals(""))return false;  
    else if(type.equalsIgnoreCase("Recipe")||type.equalsIgnoreCase("Product")) {return  
true;}
```

Replace this if-then-else statement by a single return statement.

```
public boolean checkType(String type) {  
    return type.equalsIgnoreCase("Recipe")||type.equalsIgnoreCase("Product");  
}  
  
public boolean checkMsg(String msg) {  
    return (!msg.equals(""));  
}  
  
public boolean checkNullOrEmpty(String str){  
    return (!(str == null || str.equals("")));  
}
```

So, we fix it on
different functions:

----- for this bug, a block of
code is not being properly
executed conditionally. Only
the first line of the 3-line
block is executed
conditionally, while the rest
are executed unconditionally.

Consistency | Not formatted

This line will not be executed conditionally; only the first line of this 3-line block will be. The rest will execute unconditionally. [🔗](#)

Multiline blocks should be enclosed in curly braces [Java:S2681](#)

Software qualities impacted: **Maintainability** 🟡

☐ Open ☒ Shams Abd Al-Aziz ☐ Code Smell ☒ Major

Where Why Activity More info

96
97
98 s12113...
99 s12113...
100 s12113...
101 s12113...
102 s12113...
103 s12113...
104 s12113...

```
if (type.equalsIgnoreCase("Product")) {  
    existFlag= checkProductName(name);  
    else if (type.equalsIgnoreCase("Recipe")) {  
        existFlag= checkRecipeName(name);  
    }  
    else {  
        existFlag= false;  
    }  
}
```

This line will not be executed conditionally; only the first line of this 3-line block will be. The rest will execute unconditionally.

Tags: cert ... +
Line affected: L105
Effort: 5 min
Introduced: 24 minutes ago
Open in IDE



So, we fix it to be like this:

Just replace the if statements by Switch statement.

```
public boolean checkName(String name) {  
  
    if (!(checkNullOrEmpty(name) && checkNullOrEmpty(type))) {  
        return false;  
    }  
  
    switch (type.toLowerCase()) {  
        case "product":  
            return checkProductName(name);  
        case "recipe":  
            return checkRecipeName(name);  
        default:  
            return false;  
    }  
}
```

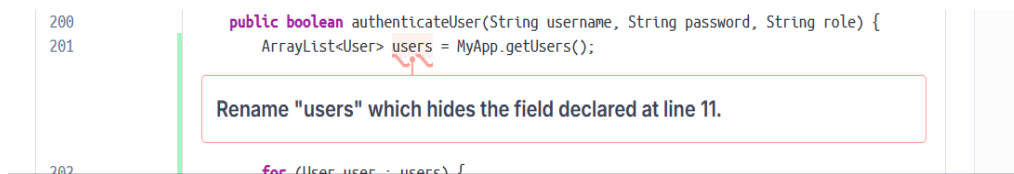
----- for this bug, it is known as the famous bug,

so, we solve it as:

```
public void printStm(String msg) {  
    logger.info(msg);  
}
```



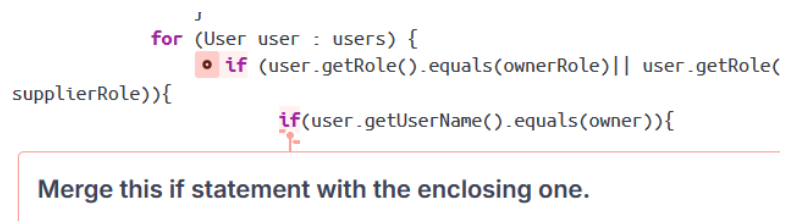
-----for this bug, we put the name of this local array list exactly as the member array list in this class, so we just re-use this member array list by calling it with their name



we fix it:

```
public boolean authenticateUser(String username, String password, String role) {
    for (User user : MyApp.getUsers()) {
        if (user.getUserName().equals(username) && user.getPassword().equals(password) && user.getRole().equals(role)) {
```

-----for this bug,



We fix it in one single if:

```
}
for (User user : users) {
    if ((user.getRole().equals(ownerRole) || user.getRole().equals(supplierRole)) && user.getUserName().equals(owner)) return user.getBranch();
}
}
```



The image above shows the rate after resolving some of the code smells.



Also, we had this issue in test class,

```
28         boolean isOwner= sweetApp.currentUser.checkRoleType(role);
29         System.out.println("Checking if user is owner: " + isOwner);
30         assertTrue(isOwner==true);
```

Use assertEquals instead.

So, we fix it:

```
System.out.println("Checking if user is owner: " + isOwner);
assertEquals(true, isOwner);
}
```

----- also, we had this bug:

```
MyApp.feedBacks=new ArrayList<FeedBack>();
MyApp.materials=new ArrayList<RawMaterial>();
```

Replace the type specification in this constructor call with the diamond operator ("<>").

So, we fix it:

```
MyApp.feedBacks=new ArrayList<>();
MyApp.materials=new ArrayList<>();
```

----for this bug, it asks us to remove all comments in our code.

```
31 //
sweetApp.currentUser.isLoggedin = sweetApp.authenticateUser(password, username, role);
```

This block of commented-out lines of code should be removed.

So, we fix it like this:

```
28 if(sweetApp.currentUser.isLoggedin==false) {sweetApp.currentUser.isLoggedin=true;}
29 s12113... assertEquals("the user must be logged in!", sweetApp.currentUser.isLoggedin, true);
);
```

Swap these 2 arguments so they are in the correct order: expected value, actual value.



★ SweetMngSys2024 / sweet project NEW PUBLIC
✓ Passed

Last analysis: 8/21/2024, 10:54 AM - 1.6k Lines of Code - Java, XML

A 0
Security

A 0
Reliability

A 121
Maintainability

A 100%
Hotspots Reviewed

85.8%
Coverage

0.0%
Duplications

also, we had this bug:

```
static public ArrayList<Massage> allMessages = new ArrayList<>();
static public ArrayList<FeedBack> feedBacks = new ArrayList<>();
```

Reorder the modifiers to comply with the Java Language Specification.

the fix:

```
public static ArrayList<FeedBack> feedBacks = new ArrayList<>();
public static ArrayList<RawMaterial> materials = new ArrayList<>();
```

Also, we had this:

```
797
798 s12113...
799 s12113...
return;
```

Remove this redundant jump.

----for this bug,

```
try {
    double p=Double.parseDouble(price2);
```

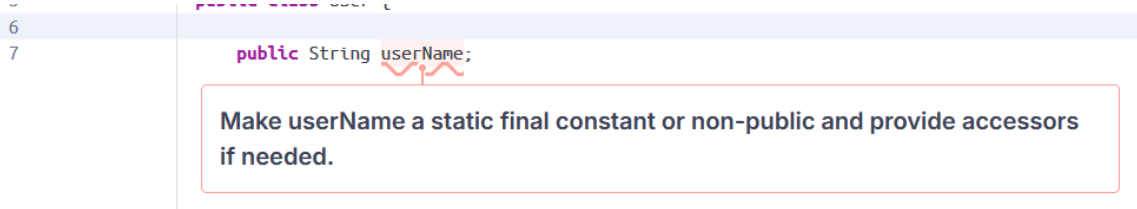
Immediately return this expression instead of assigning it to the temporary variable "p".

So, we fix it as:

```
else {
    try {
        return Double.parseDouble(price2);
```



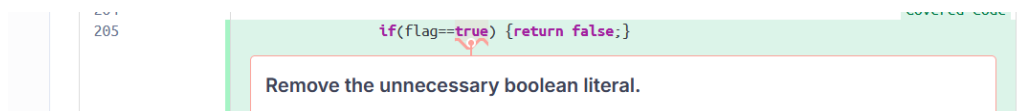

-----for this bug:



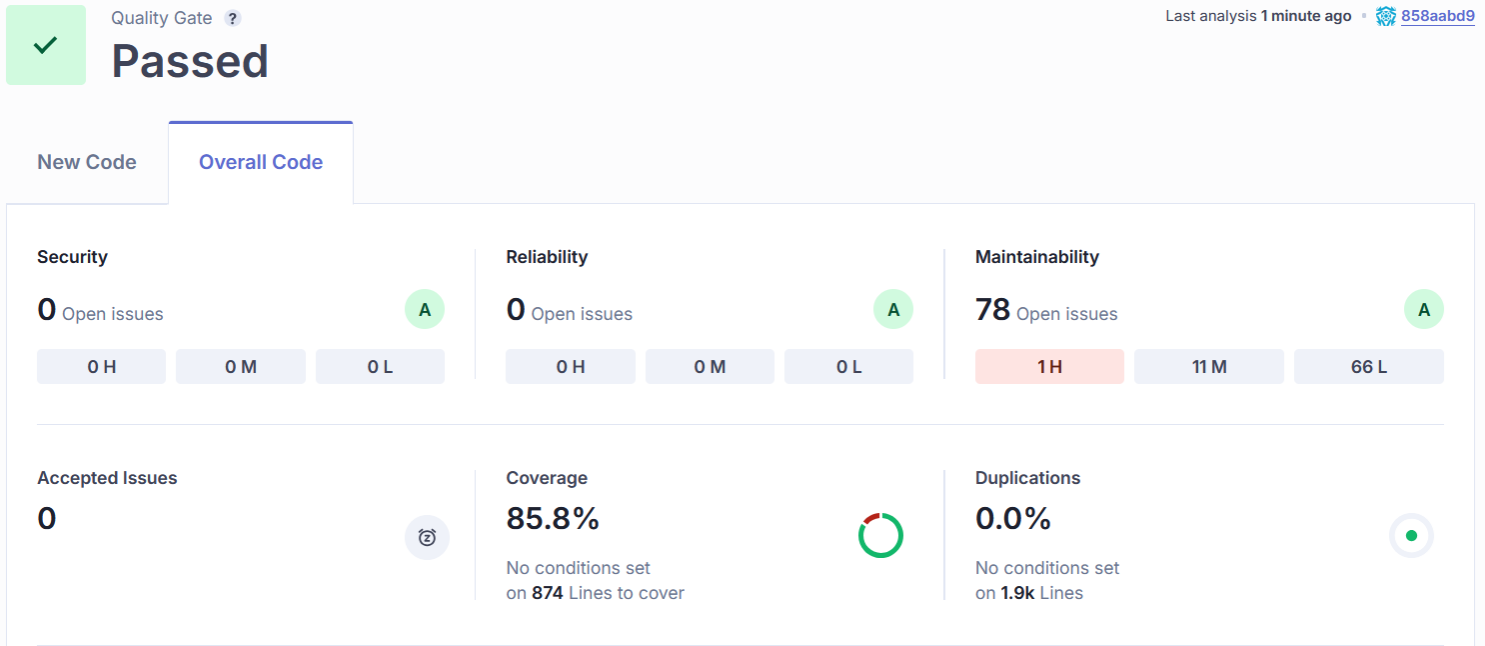
we fix it by change the member type to private and make a setter and getter to get and set it in indirect way:

```
public class User {  
    private String userName;  
  
    public String getUserName() {  
        return userName;  
    }  
  
    public void setUserName(String userName) {  
        this.userName = userName;  
    }  
}
```

also, we
had this:



So, this is the last analysis for our code after refactoring:



And this is our commits:

Course: هندسة البرمجيات Workflow runs · SweetMngSys2024

https://github.com/SweetMngSys2024/Sweet_Mng_Sys/actions

SweetMngSys2024 / Sweet_Mng_Sys

Code Issues Pull requests Actions Projects Security Insights Settings

Actions

New workflow

All workflows

SonarCloud

Management

Caches

Attestations

Runners

All workflows

Showing runs from all workflows

67 workflow runs

	Event	Status	Branch	
Update User.java	SonarCloud #62: Commit 858aabd pushed by ShamsAziz	main	15 hours ago	1m 3s
Update ownerMonetorSalesTest.java	SonarCloud #61: Commit 25b8da7 pushed by ShamsAziz	main	15 hours ago	1m 14s
Update Recipe.java	SonarCloud #60: Commit 4ac2e28 pushed by ShamsAziz	main	16 hours ago	1m 15s
Update User.java	SonarCloud #59: Commit c2327f2 pushed by ShamsAziz	main	16 hours ago	1m 12s
Update MyApp.java	SonarCloud #58: Commit a327907 pushed by ShamsAziz	main	19 hours ago	1m 20s
Update AdminCreateAccount.java	SonarCloud #57: Commit fa0dad6 pushed by ShamsAziz	main	19 hours ago	1m 8s

2:42 P 8/22/20



SonarCloud #62: Commit 858aabd pushed by ShamsAziz

So, this our code analysis, with coverage percentage in each class

The screenshot displays the SonarCloud web interface for a project named 'sweet project'. The interface is in dark mode and shows various tabs for project analysis. The 'Code' tab is selected, displaying a table of code quality metrics for different files in the project.

	Lines of Code	Security	Reliability	Maintainability	Security Hotspots	Coverage	Duplications
main/java/sweet/app	1,482	0	0	76	0	85.8%	0.0%
FeedBack.java	72	0	0	0	0	84.7%	0.0%
Message.java	46	0	0	0	0	76.3%	0.0%
MyApp.java	767	0	0	64	0	83.3%	0.0%
orders.java	80	0	0	1	0	81.8%	0.0%
product.java	164	0	0	2	0	96.0%	0.0%
RawMaterial.java	19	0	0	0	0	75.0%	0.0%
Recipe.java	93	0	0	1	0	93.5%	0.0%
User.java	241	0	0	8	0	94.7%	0.0%

8 of 8 shown

Final, using this analyzer tool helps us to learn how to refactor our code and improve it, to reuse it later or to be readable and maintainability and improve almost of the quality matrices.